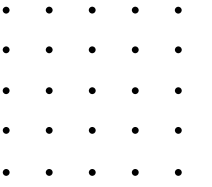
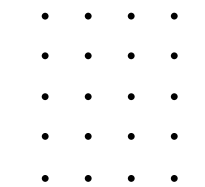




CONTULIANO BRAVO Samy, BENZIANI Naim, THEISEN Mike,
TSHISWAKA Daniel



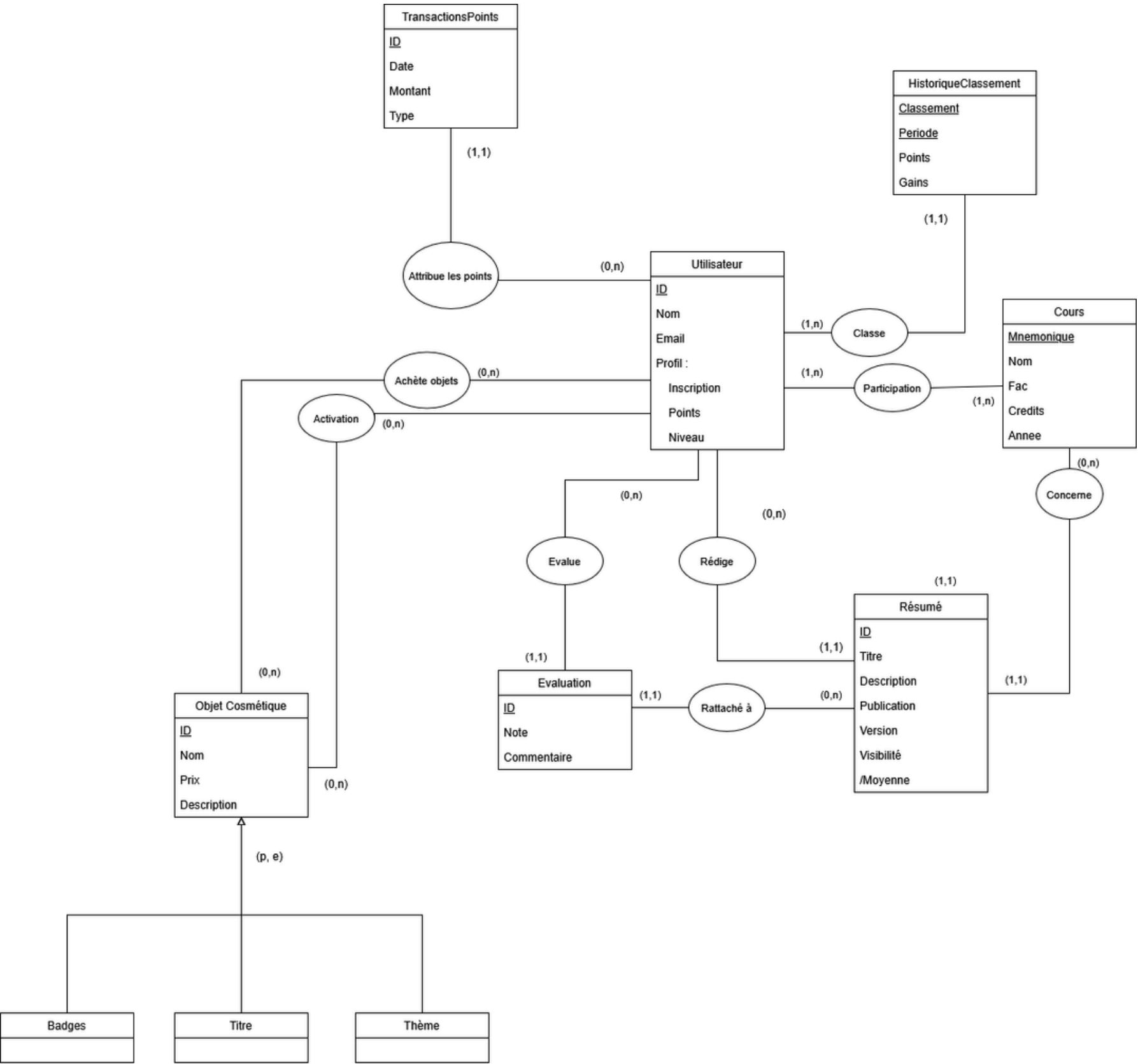
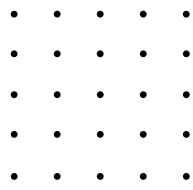
Base de données

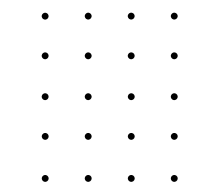


Modèle relationnel



Modèle relationnel

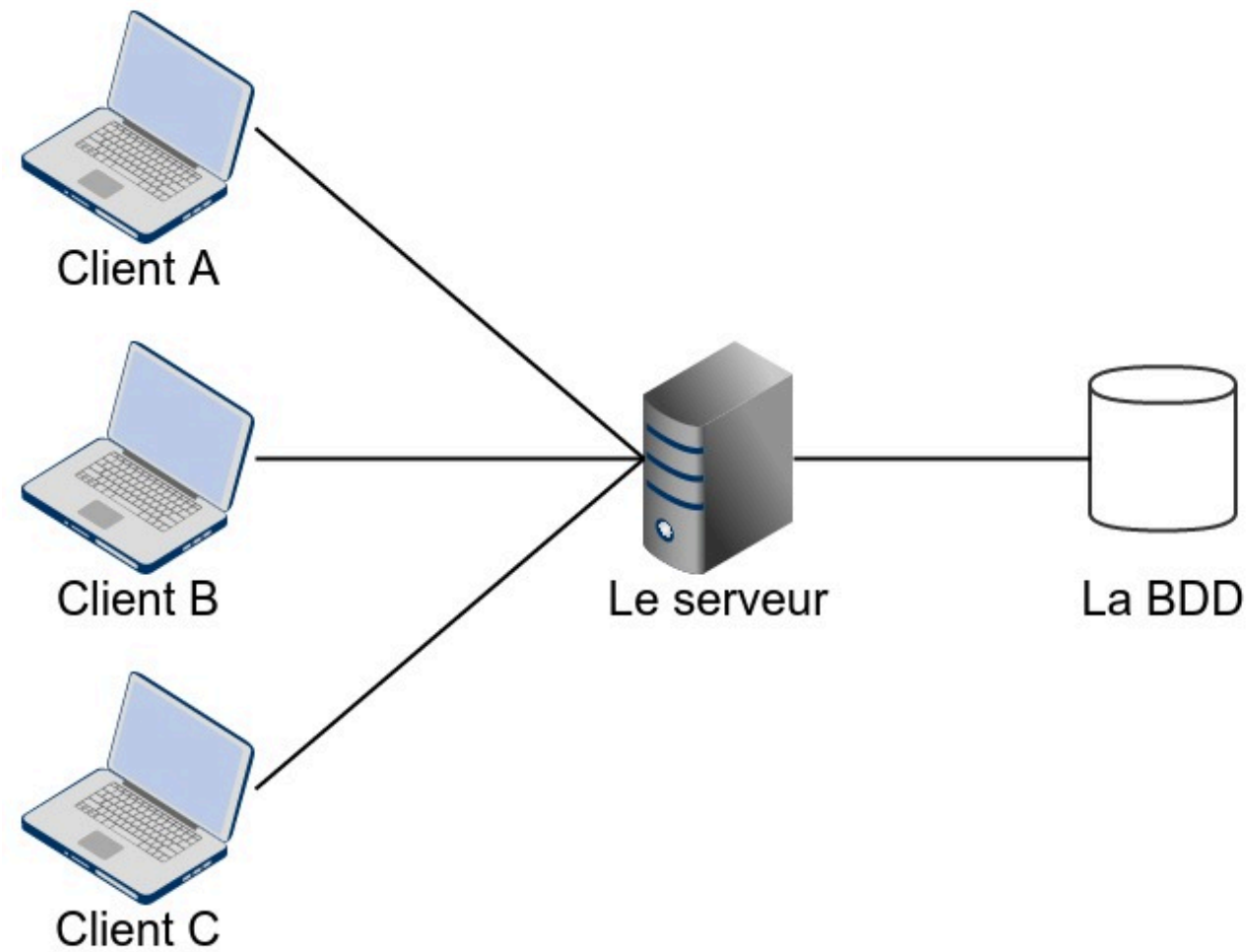
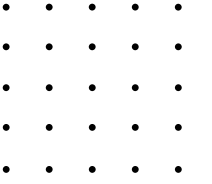




Architecture de l'application



Architecture client-serveur



Coté client:

- Interface graphique
- Vues spécialisées par fonctionnalité (Shop, Résumés, Profil...)
- Communication via ClientNetworkManager

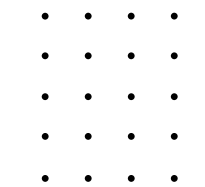
Coté serveur:

- Serveur non-bloquant (un seul thread)
- Dispatch des requêtes via ServerNetworkManager
- Accès base de données via DatabaseManager
- Base de données MySQL

Communication:

- Protocole maison : TCP + JSON
- Format : {"protocol": "...", "data": {...}}

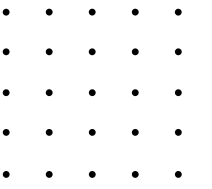




Choix technique



Choix technique



Python + customtkinter

- **CustomTkinter :**
 - interface graphique plus moderne
 - séparation des fichiers en vue pour une meilleur compréhension
- **Python :** langage moderne

Serveur non-bloquant

- **Légereté** en terme de ressource

Client-Serveur centralisée

- **Sécurité:** serveur gère la logique

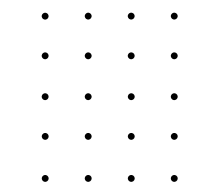
Sockets TCP bruts avec JSON

- **Socket :** échange bidirectionnel
- **TCP :** garantie l'ordre et la fiabilité des messages
- **JSON :** léger et facilement compréhensible par python

Utilisation de MySQL

- **Facilité** d'installation et de configuration
- **Grande vitesse** d'exécution sur les opérations de lecture simples
- **Rem:** PostgreSQL compatible avec le format JSON

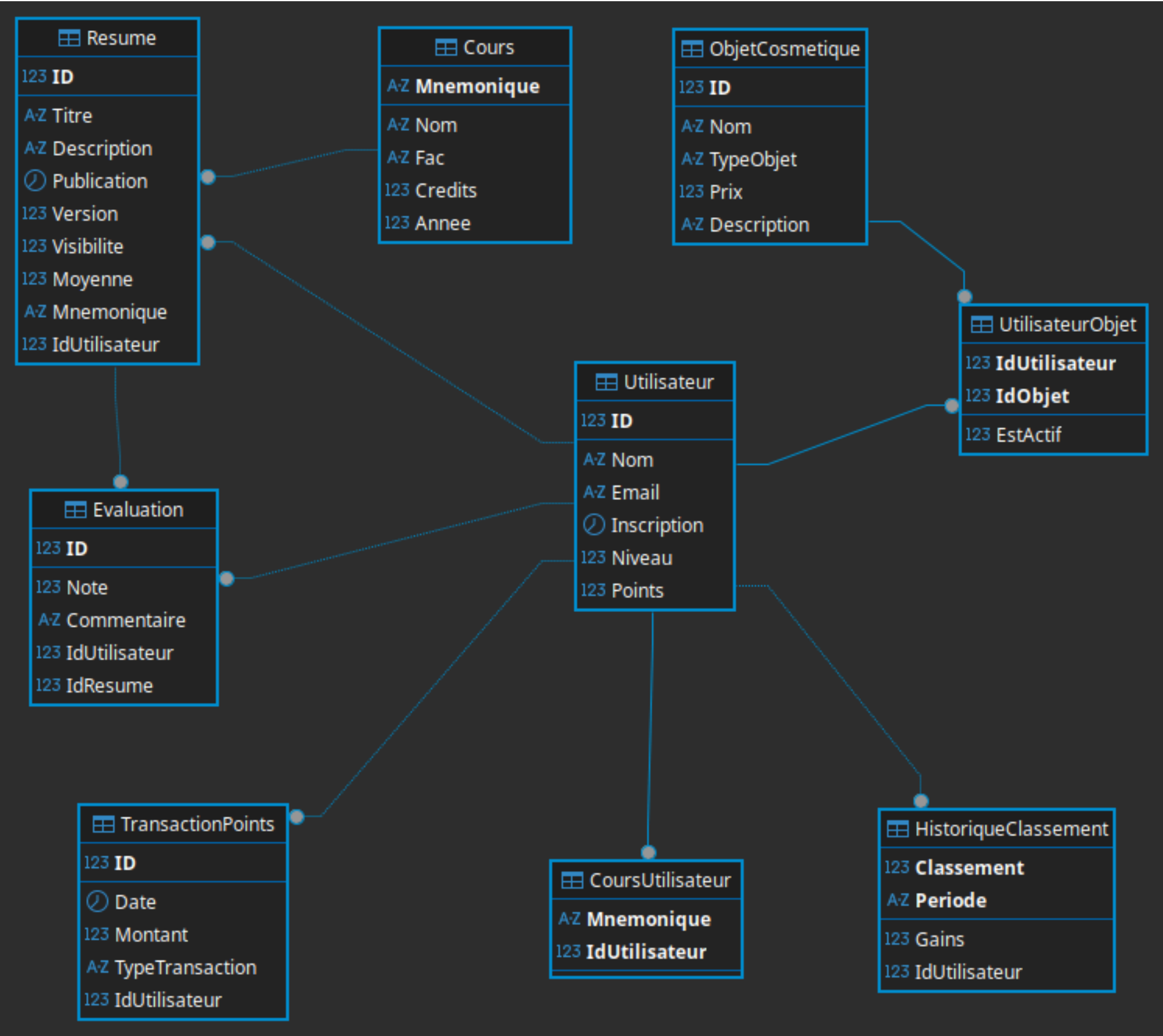
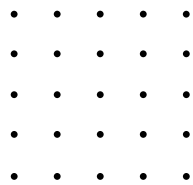




Les requêtes



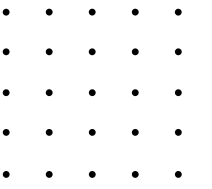
Rappel des tables



Légende:

- **123:** INT NOT NULL
- **A-Z:** VARCHAR(255)
- **Horloge:** Date
- **Tableau:** Table

Les "tops"



Les 10 utilisateurs ayant le plus de points

```
SELECT Nom
FROM Utilisateur
ORDER BY Points DESC
LIMIT 10;
```

Algèbre relationnel et calcul tuple:

Impossible car il n'y a pas de relation d'ordre dans les ensembles

Explication:

Récupération des 10 premiers utilisateurs de la table Utilisateur pour laquelle les éléments ont été triés en fonction de leur points de manière décroissante

L'objet cosmétique le plus acheté

```
SELECT o.Nom, COUNT(uo.IdUtilisateur) AS compteur
FROM UtilisateurObjet uo
JOIN ObjetCosmetique o ON uo.IdObjet = o.ID
GROUP BY o.Nom
ORDER BY compteur DESC
LIMIT 1;
```

Algèbre relationnel et calcul tuple:

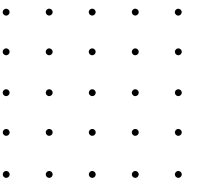
Impossible car par d'ordre dans l'ensemble, pas de limite, de groupement ou d'agregation.

Explication:

Après la jointure de la table Utilisateur et UtilisateurObjet, on a trié les cosmétiques en fonction du nombre d'occurrence dans la table d'objets achetés.



Les agrégations et moyennes



**Utilisateurs actifs dans
au moins 3 cours
différents**

```
SELECT u.Nom
FROM Utilisateur u, Resume r
WHERE u.ID = r.IdUtilisateur
GROUP BY u.ID
HAVING COUNT(DISTINCT r.Mnemonique) >= 3;
```

**Le cours ayant le plus de
résumés**

```
SELECT r.Mnemonique, COUNT(r.Mnemonique) AS Number
FROM Resume r
GROUP BY Mnemonique
HAVING COUNT(Mnemonique) = (
    SELECT MAX(myCount)
    FROM (
        SELECT Mnemonique, COUNT(Mnemonique) as myCount
        FROM Resume
        GROUP BY Mnemonique
    ) as temp
);
```

**Le nombre moyen de
résumés par utilisateur**

```
SELECT AVG(compteur)
FROM (
    --Nombre de publication par utilisateur
    SELECT r.IdUtilisateur, COUNT(r.Mnemonique) AS compteur
    FROM Resume r
    GROUP BY r.IdUtilisateur
) AS table_temporaire;
```

Algèbre relationnel

$\pi_{Nom} (\sigma_{COUNT(DISTINCT Mnemonique) \geq 3} (Utilisateur \bowtie_{u.ID=IdUtilisateur} Resume))$

Calcul tuple

$\{t.Nom \mid Utilisateur(t) \wedge \exists r_1 \exists r_2 \exists r_3 (Resume(r_1) \wedge Resume(r_2) \wedge Resume(r_3) \wedge r_1.IdUtilisateur = t.ID \wedge r_2.IdUtilisateur = t.ID \wedge r_3.IdUtilisateur = t.ID \wedge r_1.Mnemonique \neq r_2.Mnemonique \wedge r_1.Mnemonique \neq r_3.Mnemonique \wedge r_2.Mnemonique \neq r_3.Mnemonique)\}$

Explication:

1. groupe par utilisateur
2. Filtre ceux dont le nbr de mnémo distincts ≥ 3
3. Retourne ces utilisateurs

Algèbre relationnel et calcul tuple:

Impossible car pas d'ordre dans l'ensemble, pas de limite, de groupement ou d'agregation.

Explication:

1. utilise sous requete pour trouver le max
2. garde les cours atteignant ce max
3. retourne ce cours

Algèbre relationnel et calcul tuple:

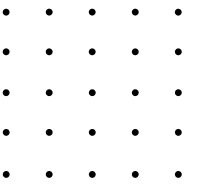
Impossible car pas d'ordre dans l'ensemble, pas de limite, de groupement ou d'agregation.

Explication:

1. compte les résumés par utilisateur dans une table temp
2. applique AVG sur les compteurs
3. retourne retourne cette moyenne



Identification des exceptions



Les utilisateurs n'ayant jamais publié de résumé

```
SELECT u.ID, u.Nom
FROM Utilisateur u
WHERE NOT EXISTS(
  SELECT r.IdUtilisateur
  FROM Resume r
  WHERE r.IdUtilisateur=u.ID);
```

Algèbre relationnel

$\pi_{Nom}(Utilisateur) - \pi_{Nom}(Utilisateur \bowtie_{ID=IdUtilisateur} Resume)$

Calcul tuple

$\{t.Nom \mid Utilisateur(t) \wedge \neg \exists r (Resume(r) \wedge r.IdUtilisateur = t.ID)\}$

Explication:

retourne les utilisateurs pour lesquels il n'existe aucun tuple dans Resume les référençant. On utilise NOT EXISTS pour vérifier cette absence.

Les utilisateurs ayant dépensé plus de points qu'ils n'en ont disponibles

```
SELECT u.Nom, u.Points, SUM(t.Montant) AS TotalDepense
FROM Utilisateur u
JOIN TransactionPoints t ON t.IdUtilisateur = u.ID
WHERE t.TypeTransaction = 'dépense'
GROUP BY u.ID, u.Nom, u.Points
HAVING SUM(t.Montant) > u.Points;
```

Algèbre relationnel et calcul tuple:

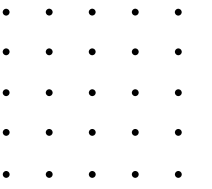
Impossible car pas d'ordre dans l'ensemble, pas de limite, de groupement ou d'agregation.

Explication:

1. groupe par utilisateur
2. filtre avec HAVING SUM > Points
3. retourne les utilisateurs dont la somme totale des dépenses en TranscationPoints dépasse leur solde actuel de points



Meilleurs résumés par cours



```
SELECT r.Mnemonique, r.Titre, r.ID, AVG(e.Note) AS note_moyenne
FROM Evaluation e
JOIN Resume r ON r.ID = e.IdResume
GROUP BY r.Mnemonique, r.Titre, r.ID
HAVING note_moyenne = (
  -- Selection le max pour chaque résumé par COURS
  SELECT MAX(table_temp.note_moyenne)
  FROM (
    -- Moyenne par résumé
    SELECT r2.Mnemonique, r2.ID, AVG(e2.Note) AS note_moyenne
    FROM Evaluation e2
    JOIN Resume r2 ON r2.ID = e2.IdResume
    GROUP BY r2.Mnemonique, r2.ID
  ) AS table_temp
WHERE table_temp.Mnemonique = r.Mnemonique
);
```

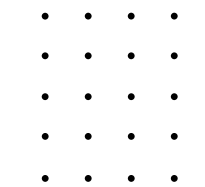
Algèbre relationnel et calcul tuple:

Impossible car cette requête nécessite des agrégations (AVG, MAX) et un groupement, qui dépassent la puissance expressive de la logique du premier ordre.

Explication:

Génère une table temporaire avec toutes les moyennes des résumés de tous les cours. Ensuite, on récupère la moyenne maximale pour le cours concerné. Pour finir, on récupère les résumés dont la note est égale au maximum qu'on a récupéré juste avant pour le cours.

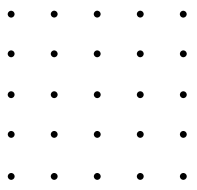




Fonctionnalité de l'app



Authentication: Connexion et inscription via nom d'utilisateur et adresse mail



Profil: Affichage des informations personnelles, points, niveau et objet équipé

Résumés: Publication, modification, suppression de résumés par cours. Consultation des résumés des autres utilisateurs

Gestion des cours: Consultation de tous les cours disponibles et inscription à ces propres cours

Evaluation: Notation et commentaire des résumés publiés par les utilisateurs.

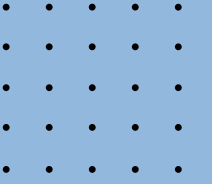
Boutique: Achat d'objets cosmétiques avec les points accumulés. Activation et désactivation des objets possédés.

Leaderboard: Classement général des utilisateurs par points et plus encore.

Historique de Transactions: Vu sur les dépenses et les gains

Système de niveau: Le niveau d'un utilisateur est défini en fonction du nombre de points qu'il possède





FAQ

